

INTERNAL RECRUITMENT SYSTEM (IJP)

Spring Boot Microservices & Angular Full-Stack Project
Complete Technical Architecture, Presentation & Interview Guide

Project Domain: Internal Recruitment & Talent Mobility

Architecture Pattern: Microservices Architecture (Database-per-Service)

Backend Stack: Java 17/24, Spring Boot 3.2.5, Spring Cloud Gateway, Eureka Server, OpenFeign, JPA, Hibernate

Frontend Stack: Angular 17+ Standalone Components, TypeScript, RxJS, HTML5, Vanilla CSS3 (LinkedIn Theme)

Database Engine: MySQL 8.0 (job_db, candidate_db, admin_db)

API Gateway Entrypoint: http://localhost:8080

Service Registry: Eureka Server (http://localhost:8761)

PROJECT DEVELOPMENT & PRESENTATION TEAM

Team Member Name	Primary Presentation Focus	Target Time
Aditya Vikram Singh	System Architecture, API Gateway, Eureka Discovery & Core Java	18–20 Mins
Khusboo Yadav	Candidate Microservice, Employee Registration, Login & Validation	18–20 Mins
Naman Kapoor	Job Service, Admin Service, Notifications, Interview & Demo	18–20 Mins

Date of Document Compilation: August 2026

Target Presentation Duration: ~60 Minutes Total (18-20 Mins Per Member + Live Demo + Viva Q&A;)

Document Purpose: Technical Defense, Architecture Master Guide, Presentation Script & 100+ Interview Questions

1. Executive Summary & Project Overview

The **Internal Job Posting (IJP) System** (also known as the *Internal Recruitment System*) is an enterprise-grade full-stack microservices application engineered to streamline talent mobility and internal recruitment within an organization.

Unlike public job search platforms (such as Indeed, LinkedIn Jobs, or Naukri.com) which connect external job seekers with hiring organizations, an **Internal Job Posting (IJP) System** operates strictly within company boundaries. It allows existing company employees to explore internal vacancies, apply for promotions or lateral team transfers, track application status updates, receive interview schedules, and interact with HR administrators.

1.1 Key Business Objectives & Real-World Problem Solved

Real-World Problem: In traditional enterprises, internal recruitment relies heavily on manual email threads, spreadsheets, and word-of-mouth recommendations. This leads to severe operational bottlenecks: high candidate drop-off, lack of transparency in application status, accidental scheduling conflicts, and high external hiring costs when qualified internal talent goes unnoticed.

Core Objectives of the IJP System:

- **Employee Retention & Career Growth:** Empowers employees to advance within the company rather than seeking external career opportunities.
- **Automated Recruitment Lifecycle:** Digitalizes job posting, employee application submission, admin shortlisting, selection/rejection, and interview scheduling.
- **Strict Employee Identity Integrity:** Enforces single-employee account identity via company emails (@company.com) and unique Employee IDs.
- **Personalized Real-Time Notifications:** Delivers isolated in-app notification alerts regarding application updates and interview schedules.
- **Decoupled Microservice Architecture:** Utilizes Spring Cloud components to ensure independent service scaling, fault tolerance, and modular maintenance.

1.2 Functional & Non-Functional Requirements

Requirement Type	Requirement Specification Details
Employee Authentication	Allows employee login using company email (@company.com) and password strictly. Employee ID alone cannot authenticate login.
Employee Registration	Allows new employees to register with First Name, Last Name, Date of Birth, Employee ID, Email (@company.com), and matching passwords. Rejects duplicate emails and duplicate Employee IDs.
Admin Authentication	Dedicated Admin login using company email (@company.com) and password. Grants access to HR Admin Dashboard.
Job Posting Management	Admins can post new internal openings, view open postings, update job status (OPEN/CLOSED), and delete job postings safely.
Job Application Flow	Employees can view open internal positions and apply. Enforces duplicate-application prevention for the same job while allowing multi-job applications for different postings.
Applicant Management	Admins can view applicants per job, update status (APPLIED, SHORTLISTED, INTERVIEW_SCHEDULED, SELECTED, REJECTED), and delete specific candidate records with cascade notification cleanup.
Interview Scheduling	Admins can schedule ONLINE (Google Meet link) or OFFLINE (Meeting Room location) interviews with automatic notification generation.
Personalized Notifications	Delivers strictly isolated in-app notifications for the logged-in employee. Shows unread badges and allows marking notifications as read without cross-employee data exposure.
Designation Master Data	Admins can create, list, and activate/deactivate master job designations.
Non-Functional Criteria	Microservice independence, Database-per-service isolation, API Gateway routing, Eureka service discovery, fast page loads (<1s API responses), clean bright UI styling.

2. Technology Stack & Deep-Dive Analysis

Every technology chosen for the IJP project serves a specific architectural or business purpose. Below is the detailed breakdown of the complete technology stack.

Layer	Technology	Version / Component	Role in IJP Project
Frontend	Angular	17+ (Standalone)	SPA UI framework handling routing, components, & forms
Frontend	TypeScript	5.x	Strongly-typed logic, models, services, & Angular components
Frontend	RxJS & HttpClient	7.8+	Asynchronous HTTP calls, Observables, & state streams
Frontend	Vanilla CSS3 & Bootstrap 5	CSS Custom Props	LinkedIn-inspired corporate theme (#0A66C2, #0B2A4A, #F7F9FC)
Backend	Java JDK	17 / 24	Core programming language for microservice business logic
Backend	Spring Boot	3.2.5	Microservice framework with embedded Tomcat server & auto-config
Microservices	Spring Cloud Gateway	4.1.x	API Gateway routing endpoint running on port 8080
Microservices	Eureka Server	Spring Cloud Netflix	Service discovery registry server running on port 8761

Microservices	OpenFeign Client	Spring Cloud OpenFeign	Declarative REST client for candidate-service -> job-service calls
Persistence	Spring Data JPA & Hibernate	Jakarta Persistence 3.1	ORM mapping Java entities to MySQL database tables
Database	MySQL Server	8.0	Relational DB schemas: job_db, candidate_db, admin_db
Build Tool	Apache Maven	3.9.16	Dependency management and multi-jar packaging tool

2.1 Technology Explanation & Sample Interview Q&A;

Technology Analysis: Spring Boot 3.2.5

- 1. What it is:** An open-source Java framework used to build stand-alone, production-grade Spring applications with minimal configuration.
- 2. Why we used it:** Eliminates complex XML configurations, bundles an embedded Tomcat web server, and provides opinionated 'Starter' dependencies.
- 3. Where used in IJP:** Used to build all three business microservices (job-service, candidate-service, admin-service) and infrastructural services.
- 4. Problem solved:** Solves deployment complexity by producing self-contained executable JAR files containing all server dependencies.

Interviewer Question: What is Spring Boot and how does it differ from traditional Spring MVC?

Trainee Answer: Spring Boot is an extension of Spring framework that eliminates boilerplate XML configuration using auto-configuration and embedded Tomcat. Traditional Spring MVC requires setting up external WAR deployment, web.xml, and manual bean definitions.

Technology Analysis: Spring Cloud Gateway

- 1. What it is:** An API Gateway built on top of Spring Framework 5, Project Reactor, and Spring Boot 2/3 providing dynamic routing, monitoring, and security.
- 2. Why we used it:** Acts as a single reverse-proxy endpoint running on port 8080, masking internal ports 8081, 8082, and 8083 from the browser.
- 3. Where used in IJP:** Configured in api-gateway microservice application.yml using path routes (/api/jobs/**, /api/candidates/**, /api/admin/**).
- 4. Problem solved:** Solves CORS security issues, centralizes client request routing, and prevents Angular from having to manage multiple backend URLs.

Interviewer Question: Why do we need an API Gateway in microservices instead of letting Angular call services directly?

Trainee Answer: Direct client-to-service communication forces frontend code to hardcode dynamic microservice ports, exposes internal infrastructure, and requires configuring CORS headers across every backend service. API Gateway centralizes entry, routing, and security at port 8080.

Technology Analysis: Spring Cloud Eureka (Service Discovery)

- 1. What it is:** A REST-based service registry server developed by Netflix and integrated into Spring Cloud for discovering microservice instances dynamically.
- 2. Why we used it:** Enables microservices to register dynamically upon startup without hardcoding IP addresses or port numbers in configuration files.
- 3. Where used in IJP:** Running in service-discovery module on port 8761. Microservices specify `spring.application.name` and register automatically.
- 4. Problem solved:** Solves network location rigidity in distributed cloud environments where services may restart on different hostnames/ports.

Interviewer Question: What happens if Eureka Server crashes while microservices are running?

Trainee Answer: Eureka clients cache registry information locally. If Eureka crashes, microservices can still communicate using cached endpoints for a period. However, newly started microservices will be unable to register until Eureka recovers.

Technology Analysis: Spring Data JPA & Hibernate

- 1. What it is:** Spring Data JPA is an abstraction layer that provides pre-built CRUD methods. Hibernate is an Object-Relational Mapping (ORM) framework.
- 2. Why we used it:** Allows developers to interact with MySQL databases using Java objects (Entities) instead of writing raw JDBC queries.
- 3. Where used in IJP:** Used across all repositories (CandidateRepository, JobPostingRepository, AdminRepository, NotificationRepository, InterviewRepository).
- 4. Problem solved:** Eliminates SQL boilerplate code, handles database connections safely, and automatically updates database tables via `ddl-auto=update`.

Interviewer Question: What is the difference between JPA and Hibernate?

Trainee Answer: JPA (Jakarta Persistence API) is a standard Java specification that defines rules for ORM. Hibernate is an actual implementation of the JPA specification that generates SQL queries under the hood.

Technology Analysis: Angular 17+ Standalone Components

- 1. What it is:** A modern TypeScript-based web application framework developed by Google for building single-page applications (SPAs).
- 2. Why we used it:** Standalone components remove the need for NgModule declarations, making frontend code modular, lightweight, and easy to maintain.
- 3. Where used in IJP:** Powers the entire `ijp-frontend` application running on port 4200 (Home, Login, Register, Apply, Dashboards, Notifications).
- 4. Problem solved:** Solves full-page reload slowness by dynamically updating page DOM elements via RxJS Observables and Angular Routing.

Interviewer Question: What is a Single Page Application (SPA) and how does Angular handle navigation?

Trainee Answer: An SPA loads a single HTML page (`index.html`) and dynamically rewrites page sections as the user interacts. Angular Router intercepts URL clicks and swaps component views without sending requests for new HTML pages from the server.

3. System Architecture & Request Flows

The IJP System implements a **Decoupled Microservices Architecture** following the **Database-per-Service Pattern**. Below is the architectural diagram showing how components interact.



3.1 Microservices vs. Monolithic Architecture Trade-Offs

Why we chose Microservices over Monolith:

- 1. **Service Independence & Loose Coupling:** candidate-service, job-service, and admin-service are separate projects. Changes in job posting logic do not affect candidate registration code.
- 2. **Independent Scalability:** During a major internal hiring drive, candidate-service receives high traffic. It can be scaled independently without duplicating job-service instances.
- 3. **Fault Isolation:** If admin-service fails, employees can still view open job postings and submit candidate applications.
- 4. **Autonomous Data Storage:** Database-per-service prevents accidental table locking and cross-domain SQL join dependencies.

Disadvantages & Trade-offs of Microservices:

- **Distributed System Complexity:** Managing multiple application instances, network routing, and service ports.
- **Network Latency:** Inter-service communication via REST/Feign adds slight network delay compared to in-memory method calls.
- **Data Consistency:** Distributed databases cannot perform simple ACID SQL joins across schemas; consistency requires application validation.

3.2 Step-by-Step Inter-Service Communication Flow

Example Flow: Candidate Applies for an Internal Job Posting

- 1. **User Action:** Employee fills application form on Angular frontend (`ApplyComponent`).
- 2. **HTTP Call:** Angular HttpClient sends `POST /api/candidates` with candidate JSON payload to Gateway (`http://localhost:8080`).
- 3. **Gateway Routing:** API Gateway inspects route `/api/candidates/\*\*` and proxies request to `CANDIDATE-SERVICE` on port `8082`.
- 4. **Local Validation:** `CandidateService` verifies company email ends with `@company.com` and checks duplicate application constraints.
- 5. **Inter-Service Call (Feign Client):** `CandidateService` invokes `JobServiceClient.getJobById(jobId)` via OpenFeign.

6. **Feign Gateway Routing:** Feign queries Eureka Server (:8761), resolves `JOB-SERVICE` (:8081), and executes `GET /api/jobs/{id}`.
7. **Job Status Check:** `CandidateService` checks if `job.getStatus()` is `OPEN`. If closed, throws `RuntimeException('Cannot apply: Job is CLOSED!')`.
8. **Persistence:** Candidate application record is saved into `candidate\_db.candidate` table with status `APPLIED`. Response returned to Angular.

4. Detailed Microservice Breakdown

4.1 Service Discovery Microservice (Eureka Server)

Module Name: `service-discovery` | **Port:** `8761` | **Main Class:** `ServiceDiscoveryApplication`

The Service Discovery microservice acts as the central telephone directory for all microservices in the system. Upon startup, microservices send registration requests containing their application name, host, and port.

```
# service-discovery/src/main/resources/application.yml server: port: 8761 eureka: instance: hostname: localhost client: register-with-eureka: false # Server does not register with itself fetch-registry: false # Server does not fetch registry from elsewhere service-url: defaultZone: http://${eureka.instance.hostname}:${server.port}/eureka/
```

Key Concept - Heartbeats & Renewals: Registered microservices send heartbeat ping signals every 30 seconds to Eureka. If Eureka fails to receive heartbeats for 90 seconds, it evicts the instance from the registry.

4.2 API Gateway Microservice (Spring Cloud Gateway)

Module Name: `api-gateway` | **Port:** `8080` | **Main Class:** `ApiGatewayApplication`

Spring Cloud Gateway serves as the single reverse-proxy endpoint for all client traffic. It dynamically routes incoming HTTP requests to destination microservices using Eureka service names.

```
# api-gateway/src/main/resources/application.yml server: port: 8080 spring: application: name: api-gateway cloud: gateway: discovery: locator: enabled: true lower-case-service-id: true routes: - id: job-service uri: lb://JOB-SERVICE predicates: - Path=/api/jobs/** - id: candidate-service uri: lb://CANDIDATE-SERVICE predicates: - Path=/api/candidates/** - id: admin-service uri: lb://ADMIN-SERVICE predicates: - Path=/api/admin/**
```

4.3 Candidate Service Microservice

Module Name: `candidate-service` | **Port:** `8082` | **Database:** `candidate\_db`

The Candidate Service is the core operational module managing employee candidate profiles, registration, email-only authentication, job applications, interview schedules, and personalized in-app notifications.

Core Features Implemented in Candidate Service:

- **Employee Registration:** Accepts candidate data, validates `@company.com` email format, rejects duplicate emails, rejects duplicate employee IDs, and saves record with initial status `APPLIED`. (Endpoint: `POST /api/candidates/register`).
- **Email-Only Authentication:** Authenticates employee using company email (`email`) and password. Employee ID alone is NOT accepted as a login identifier. (Endpoint: `POST /api/candidates/login`).
- **Candidate Application Management:** Enforces single-candidate profile across multiple job applications while preventing duplicate applications for the same job posting.
- **Cascade Employee Deletion:** Admin candidate deletion (`DELETE /api/candidates/{id}`) performs transactionally safe deletion of candidate applications, associated notifications, and interview schedules.
- **Personalized Notifications:** Resolves all candidate IDs associated with the logged-in employee email/empId and returns strictly isolated notifications (`GET /api/candidates/notifications/employee/{identifier}`).

4.4 Job Service Microservice

Module Name: `job-service` | **Port:** `8081` | **Database:** `job\_db`

Manages internal job vacancy listings created by HR Admins. Provides endpoints for fetching open jobs (`GET /api/jobs/open`), creating job postings (`POST /api/jobs`), updating status (`PUT /api/jobs/{id}/status`), and deleting postings (`DELETE /api/jobs/{id}`).

4.5 Admin Service Microservice

Module Name: `admin-service` | **Port:** `8083` | **Database:** `admin\_db`

Manages HR Admin login authentication (`POST /api/admin/login`) and Designation Master Data (`/api/admin/designations`). Note: Admin Registration is intentionally **NOT implemented** per business rules; admin accounts exist in `admin\_db` pre-configured.

5. Database Design & JPA Model Specifications

The IJP project follows the **Database-per-Service Pattern** with three distinct MySQL databases: `job\_db`, `candidate\_db`, and `admin\_db`.

5.1 Complete Table Specifications

Database	Table Name	Columns & Constraints	JPA Entity Mapping
job_db	job_posting`	`id` BIGINT PK AUTO_INC `job_id` VARCHAR(255) NOT NULL `designation` VARCHAR(255) `description` TEXT `location` VARCHAR(255) `skill_set` VARCHAR(255) `experience` VARCHAR(255) `salary_min` DOUBLE `salary_max` DOUBLE `status` VARCHAR(50)	`JobPosting.java` `@Entity` `@Table(name='job_posting')`
candidate_db	`candidate`	`id` BIGINT PK AUTO_INC `first_name` VARCHAR(255) `last_name` VARCHAR(255) `employee_id` VARCHAR(255) `dob` VARCHAR(255) `email` VARCHAR(255) `password` VARCHAR(255) `job_id` BIGINT `status` VARCHAR(255) <i>Unique Constraints:</i> `(email, job_id)`, `(employee_id, job_id)`	`Candidate.java` `@Entity` `@Table(name='candidate',` `uniqueConstraints={...})`
candidate_db	`notification`	`id` BIGINT PK AUTO_INC `candidate_id` BIGINT `title` VARCHAR(255) `message` TEXT `type` VARCHAR(255) `is_read` BIT(1) `created_at` VARCHAR(255)	`Notification.java` `@Entity` `@Table(name='notification')`
candidate_db	`interview`	`id` BIGINT PK AUTO_INC `candidate_id` BIGINT `job_id` BIGINT `interview_date` VARCHAR(255) `interview_time` VARCHAR(255) `interviewer` VARCHAR(255) `interview_mode` VARCHAR(50) `location` VARCHAR(255) `meeting_link` VARCHAR(255) `status` VARCHAR(50)	`Interview.java` `@Entity` `@Table(name='interview')`
admin_db	`admin`	`id` BIGINT PK AUTO_INC `email` VARCHAR(255) UNIQUE `password` VARCHAR(255)	`Admin.java` `@Entity` `@Table(name='admin')`
admin_db	`designation`	`id` BIGINT PK AUTO_INC `name` VARCHAR(255) UNIQUE `status` VARCHAR(50)	`Designation.java` `@Entity` `@Table(name='designation')`

5.2 Multi-Job Applications vs Unique Identity Mapping

Crucial Interview Clarification: In our implementation, an employee profile belongs to the individual employee (identified by company email ``@company.com`` and Employee ID ``E901``).

- When Naman Dheer (``naman@company.com``) applies for Job 5, a Candidate entity is created with ``jobId = 5``.
- When Naman applies for Job 6, another Candidate entity is created with ``jobId = 6``.
- **Why are separate rows stored?** Because candidate application status (``APPLIED``, ``SHORTLISTED``, ``INTERVIEW_SCHEDULED``, ``SELECTED``, ``REJECTED``) is specific to each individual job application.
- **Duplicate Application Rule:** Applying for Job 5 a second time is rejected with *'You have already applied for this job.'* enforced by ``@UniqueConstraint(columnNames = {'email', 'jobId'})``.

6. Complete REST API Master Specification

All API calls route through Spring Cloud API Gateway at `http://localhost:8080`. Below is the complete table of all active endpoints in the system.

HTTP Method	Gateway Endpoint URL	Target Microservice	Purpose & Access Control
`POST`	`/api/candidates/register`	Candidate Service (:8082)	Registers new employee account. Rejects duplicate email/empld.
`POST`	`/api/candidates/login`	Candidate Service (:8082)	Authenticates employee using EMAIL + PASSWORD strictly.
`POST`	`/api/candidates`	Candidate Service (:8082)	Submits candidate job application. Validates OPEN status via Feign.
`GET`	`/api/candidates`	Candidate Service (:8082)	Fetches all candidate applications for Admin Dashboard.
`DELETE`	`/api/candidates/{id}`	Candidate Service (:8082)	Deletes specific candidate record & cascade deletes notifications/interviews.
`PUT`	`/api/candidates/{id}/status`	Candidate Service (:8082)	Updates applicant status (SHORTLISTED, SELECTED, REJECTED).
`POST`	`/api/candidates/interviews`	Candidate Service (:8082)	Schedules ONLINE or OFFLINE interview & sends notification.
`GET`	`/api/candidates/notifications/employee/{identifier}`	Candidate Service (:8082)	Fetches personalized in-app notifications for logged-in employee.
`GET`	`/api/candidates/notifications/employee/{identifier}/unread-count`	Candidate Service (:8082)	Returns unread notification badge count for employee.
`PUT`	`/api/candidates/notifications/{id}/read`	Candidate Service (:8082)	Marks specific notification as read.
`GET`	`/api/jobs`	Job Service (:8081)	Fetches all job postings.
`GET`	`/api/jobs/open`	Job Service (:8081)	Fetches OPEN job postings for home page & candidate apply view.
`POST`	`/api/jobs`	Job Service (:8081)	Creates new internal job posting (Admin only).
`DELETE`	`/api/jobs/{id}`	Job Service (:8081)	Deletes specific job posting record by ID.
`POST`	`/api/admin/login`	Admin Service (:8083)	Authenticates HR Admin credentials (admin@company.com / admin123).
`GET`	`/api/admin/designations`	Admin Service (:8083)	Fetches designation master data list.
`POST`	`/api/admin/designations`	Admin Service (:8083)	Creates new designation master record.

7. Team Division & Presentation Strategy (~60 Mins Total)

To deliver a seamless 1-hour team presentation, technical topics, presentation responsibilities, live demo execution, and interview focus areas are balanced equally among the three team members.

Team Member	Allocated Topics & Architectural Focus	Time	Live Demo Role	Interview Focus
Aditya Vikram Singh	- System Architecture & Microservice Rationale - API Gateway Routing & Eureka Discovery - Core Java, Maven Build & Project Setup	18–20 Mins	Architectural overview, Eureka Dashboard & API Gateway request flow demo	Microservices vs Monolith, Eureka registry, API Gateway routing, Core Java
Khusboo Yadav	- Candidate Microservice Architecture - Employee Registration & Email-Only Login - Validation, Data Isolation & Security - candidate_db JPA/Hibernate Mapping	18–20 Mins	Employee Registration flow, Email-Only Login, & Employee Dashboard demo	Authentication, Validation logic, JPA/Hibernate, SQL constraints, Security
Naman Kapoor	- Job Service & Admin Service Architecture - Job Applications, Status Updates & Deletion - Interview Scheduling & In-App Notifications - Angular Frontend Architecture & RxJS	18–20 Mins	Admin Login, Job Posting, Shortlisting, Interview Scheduling & Notifications demo	OpenFeign communication, Admin operations, Angular services/RxJS, Testing

7.1 Live Demonstration Master Plan (10-15 Minutes)

Step	Presenter	Live Demonstration Action
Step 1	Aditya	Opens Eureka Dashboard ('http://localhost:8761') & shows active service registrations ('JOB-SERVICE', 'CANDIDATE-SERVICE', 'ADMIN-SERVICE', 'API-GATEWAY').
Step 2	Khusboo	Opens Angular Frontend ('http://localhost:4200') -> Clicks 'Login' -> Shows initial empty login state -> Clicks 'Register as Employee'.
Step 3	Khusboo	Submits new registration for Priya Sharma ('priya@company.com', 'EMP707'). Shows success toast and automatic redirection to Employee Login.
Step 4	Khusboo	Attempts login with Employee ID 'EMP707' -> Shows validation error (*Login requires a valid company email address*).
Step 5	Khusboo	Logs in with 'priya@company.com' + 'Pass@707' -> Opens Employee Dashboard showing empty notifications and active applications.
Step 6	Naman	Opens new tab -> Logs in as Admin ('admin@company.com' / 'admin123') -> Displays Admin Dashboard & Job Postings table.
Step 7	Naman	Creates new job posting ('Cloud Solutions Engineer', 'JOB909', Salary Range 90k-120k). Shows job appearing immediately in table.
Step 8	Naman	Switches to Candidate Applications view -> Finds Naman Dheer ('naman@company.com') -> Clicks 'Shortlist'.
Step 9	Naman	Schedules an ONLINE interview for Naman Dheer with Google Meet link and interview date/time.
Step 10	Naman	Switches back to Employee view -> Shows personalized Notification Inbox displaying *Profile Shortlisted* and *Interview Scheduled* with unread badge counter.
Step 11	Aditya	Summarizes total system flow, database isolation, and concludes presentation.

8. Individual Presentation Scripts & Speaking Notes

8.1 Aditya Vikram Singh — Presentation Script (~18-20 Mins)

"Respected Trainers, Evaluators, and Colleagues, good morning/afternoon. My name is **Aditya Vikram Singh**, and today, alongside my team members Khusboo Yadav and Naman Kapoor, I am proud to present our project: the **Internal Job Posting (IJP) System**. To set the context, an Internal Job Posting system is an enterprise recruitment platform designed exclusively for existing company employees. In large corporate organizations, employees often seek lateral team transfers, career promotions, or new role challenges. Without a centralized digital portal, HR teams struggle with manual email threads, lost applications, and scheduling conflicts. Our project digitalizes this complete lifecycle. I will be explaining our **System Architecture, Spring Cloud API Gateway, Eureka Service Discovery, and Core Java Backend Foundations. Architectural Philosophy:** We chose a **Decoupled Microservices Architecture** following the **Database-per-Service Pattern**. Rather than building one large monolithic application where all code shares a single SQL database, we split the domain into three independent Spring Boot services: ``job-service``, ``candidate-service``, and ``admin-service``. Why microservices? First, **Independent Scalability:** During a major company hiring drive, thousands of candidates may access ``candidate-service`` simultaneously. We can scale ``candidate-service`` instances without wasting memory on ``job-service``. Second, **Fault Isolation:** If our admin reporting service undergoes maintenance, employees can still log in, browse open positions, and submit applications cleanly. **Service Discovery (Eureka Server):** In cloud microservices, service IP addresses and ports are dynamic. We run **Eureka Server** on port ``8761``. When ``candidate-service`` starts up, it registers itself with Eureka. When ``candidate-service`` needs to communicate with ``job-service`` to verify job availability, it uses an **OpenFeign REST client** (`@FeignClient(name = "job-service")`). Feign queries Eureka, resolves the target IP, and executes the HTTP request without any hardcoded URLs. **API Gateway (Spring Cloud Gateway):** Our **API Gateway** runs on port ``8080`` and acts as the single reverse-proxy endpoint for our Angular single-page application. Frontend browsers communicate strictly with port ``8080``. The Gateway inspects incoming path predicates: - ``/api/jobs/**`` routes to ``lb://JOB-SERVICE`` (port 8081) - ``/api/candidates/**`` routes to ``lb://CANDIDATE-SERVICE`` (port 8082) - ``/api/admin/**`` routes to ``lb://ADMIN-SERVICE`` (port 8083) This solves Cross-Origin Resource Sharing (CORS) centrally, masks internal backend ports from external inspection, and simplifies security. I will now hand over to Khusboo to walk through the Candidate Service and Employee Registration features."

8.2 Khusboo Yadav — Presentation Script (~18-20 Mins)

"Thank you, Aditya. Respected evaluators, my name is **Khusboo Yadav**, and I will be presenting the **Candidate Microservice Architecture, Employee Registration, Email-Only Authentication, and Data Isolation Mechanics. Candidate Microservice Overview:** Our ``candidate-service`` operates on port ``8082`` with its dedicated ``candidate_db`` database. It handles employee accounts, candidate registrations, job applications, status updates, interview schedules, and personalized in-app notifications. **Employee Registration Feature:** We implemented full employee registration via ``POST /api/candidates/register``. When a new employee joins the platform, they fill out our Angular form (``RegisterComponent``) providing First Name, Last Name, Date of Birth, Employee ID, Company Email (``@company.com``), and matching Passwords. Our backend enforces strict validation: 1. **Company Email Validation:** The email must end strictly with ``@company.com`` (case-insensitive). Non-company domains like ``@gmail.com`` are rejected immediately. 2. **Duplicate Email Protection:** Checks ``candidateRepository.findByEmailIgnoreCase(email)``. If present, rejects with *'An employee with this email already exists.'* 3. **Duplicate Employee ID Protection:** Checks ``candidateRepository.findByEmployeeId(empId)``. If present, rejects with *'An employee with this employee ID already exists.'* 4. **Default Status Assignment:** New employee profiles are initialized with status ``APPLIED`` and ``jobId = 0``. **Email-Only Authentication:** We refined Employee Login (``POST /api/candidates/login``) to accept **EMAIL ONLY** plus password. Previously, logging in via Employee ID alone was permitted, which created identity ambiguities. Now, the login service queries candidate records strictly by email (``findByEmailIgnoreCase``). Employee ID alone cannot authenticate a session. **Personalized Notifications & Data Isolation:** Data privacy is paramount. An employee must never view another employee's notifications. In ``CandidateService``, our endpoint ``GET /api/candidates/notifications/employee/{identifier}`` resolves all candidate application IDs belonging to the authenticated employee's email or Employee ID, fetching notifications strictly for those IDs. In Angular, we completely removed the employee selector dropdown from the Notifications page, binding the Inbox view directly to ``authService.currentUserValue``. I will now pass the presentation to Naman to cover Job Service, Admin Service, and demonstrate our live application."

8.3 Naman Kapoor — Presentation Script (~18-20 Mins)

"Thank you, Khusboo. Hello everyone, my name is **Naman Kapoor**. I will be covering the **Job Microservice**, **Admin Microservice**, **Interview Scheduling**, **Angular Architecture**, and leading our **Live Demonstration**. **Job Service Microservice:** Operating on port ``8081`` with ``job_db``, ``job-service`` manages the lifecycle of internal vacancies (``JobPosting`` entity). HR Admins can post new jobs (``POST /api/jobs``), view open jobs (``GET /api/jobs/open``), update status (``PUT /api/jobs/{id}/status``), and delete postings (``DELETE /api/jobs/{id}``). When a job is deleted, ``JobPostingService.deleteJob(id)`` removes the posting record from ``job_db``. Candidate application records for other jobs remain unaffected. **Admin Service & Designation Master Data:** ``admin-service`` runs on port ``8083`` with ``admin_db``. It authenticates HR Admins (``POST /api/admin/login``). It also maintains the Designation Master (``/api/admin/designations``), allowing admins to create master job titles (such as ``*Cloud Architect*`` or ``*Senior Java Engineer*``) and toggle their active/inactive status. **Interview Scheduling Lifecycle:** When an admin schedules an interview (``POST /api/candidates/interviews``), ``CandidateService`` validates whether the interview is ``**ONLINE**`` (requires Google Meet URL) or ``**OFFLINE**`` (requires Meeting Room location). Upon saving, it updates the candidate status to ``INTERVIEW_SCHEDULED`` and automatically creates a formatted notification record in ``candidate_db.notification``. **Angular Frontend & UI Design System:** Our Angular application (``ijp-frontend``) uses standalone components, RxJS Observables, and Angular Router guards (``employeeGuard``, ``adminGuard``). We designed a clean, LinkedIn-inspired professional dark blue theme (``#0A66C2`` primary blue, ``#0B2A4A`` header navbar, ``#F7F9FC`` bright page background). We will now begin our **Live System Demonstration...**"

9. Comprehensive Technical Interview & Viva Preparation (100+ Q&A;)

This section contains 100 categorized technical interview and viva questions crafted specifically around the IJP project technology stack and implementation.

[Q1 (Core Java)] What is the difference between JDK, JRE, and JVM?

Answer: JDK is the development kit containing compiler (javac) and tools. JRE is the runtime environment providing libraries. JVM is the virtual machine executing bytecode.

[Q2 (Core Java)] What is the difference between String, StringBuilder, and StringBuffer?

Answer: String is immutable. StringBuilder is mutable and non-thread-safe (fast). StringBuffer is mutable and thread-safe (synchronized).

[Q3 (Core Java)] How does HashMap work internally in Java?

Answer: HashMap uses an array of Nodes (hash buckets). It computes hashCode(), determines index via (n-1)&hash;, and handles collisions using LinkedList or Red-Black Tree (threshold > 8).

[Q4 (OOP)] What are the 4 main pillars of OOP and how are they used in IJP?

Answer: Encapsulation (private fields in Candidate entity), Inheritance (Extending JpaRepository), Polymorphism (Overloading candidate search methods), Abstraction (Interfaces for services and repositories).

[Q5 (Core Java)] What is an Exception in Java and how is custom exception handling done in Spring Boot?

Answer: Exceptions disrupt normal program flow. In Spring Boot, custom exceptions thrown in services are caught by `@ControllerAdvice` or caught in Controller try-catch blocks returning `ResponseEntity` with HTTP status codes.

[Q6 (Spring Boot)] What is Dependency Injection (DI) and Inversion of Control (IoC)?

Answer: IoC is the architectural principle where the framework manages object lifecycles. DI is the pattern where Spring injects required bean dependencies into classes using `@Autowired` or constructor injection.

[Q7 (Spring Boot)] What is the purpose of `@SpringBootApplication` annotation?

Answer: It is a combination of `@Configuration` (defines Spring beans), `@EnableAutoConfiguration` (auto-configures Spring Boot starters), and `@ComponentScan` (scans package for components).

[Q8 (Spring Boot)] What are Spring Bean Scopes?

Answer: Singleton (default, single instance per IoC container), Prototype (new instance every request), Request (per HTTP request), Session (per HTTP session).

[Q9 (Spring Boot)] What is the role of `pom.xml` in Spring Boot Maven projects?

Answer: pom.xml (Project Object Model) manages project dependencies, plugin configurations, build lifecycles, and Java compiler versions.

[Q10 (Spring Boot)] How does `application.yml` or `application.properties` work?

Answer: It externalizes application configuration (server port, database URL, credentials, Spring Cloud properties) without recompiling Java code.

[Q11 (REST API)] What is a REST API and what HTTP methods are used in IJP?

Answer: Representational State Transfer API. We use ``GET`` (fetch data), ``POST`` (create candidate/job), ``PUT`` (update status/read state), ``DELETE`` (remove candidate/job).

[Q12 (REST API)] What is the difference between `@RequestParam`` and `@PathVariable``?

Answer: `@PathVariable`` extracts values from URL path (`/api/candidates/{id}``). `@RequestParam`` extracts query parameters (`/api/jobs?status=OPEN``).

[Q13 (REST API)] What is the difference between HTTP status code 400, 401, 404, and 500?

Answer: 400 = Bad Request (validation failure), 401 = Unauthorized (login failure), 404 = Not Found (missing resource), 500 = Internal Server Error (uncaught backend exception).

[Q14 (Microservices)] What is the Database-per-Service pattern and why did we use it?

Answer: Each microservice owns a dedicated database (`job_db`, `candidate_db`, `admin_db`). It ensures loose coupling and independent database schema updates.

[Q15 (Microservices)] How does OpenFeign work in Spring Cloud?

Answer: OpenFeign is a declarative REST client. Developers define an interface with `@FeignClient(name='job-service`)`, and Spring dynamically generates the HTTP implementation.

[Q16 (JPA/Hibernate)] What is ORM (Object-Relational Mapping)?

Answer: ORM maps relational database tables to Java object classes (`@Entity``), allowing developers to query databases using Java objects instead of raw SQL.

[Q17 (JPA/Hibernate)] What is `@UniqueConstraint`` in JPA?

Answer: Defines composite table uniqueness constraints in JPA (`@Table(uniqueConstraints = {@UniqueConstraint(columnNames = {'email', 'jobId'})})``).

[Q18 (JPA/Hibernate)] What is the difference between `ddl-auto=update`` and `ddl-auto=create-drop``?

Answer: `update`` modifies table structures dynamically without deleting existing data. `create-drop`` drops tables on application shutdown, losing data.

[Q19 (MySQL)] What is a Primary Key and Foreign Key?

Answer: Primary Key uniquely identifies a table row (`@Id @GeneratedValue``). Foreign Key references a Primary Key in another table to enforce relational integrity.

[Q20 (MySQL)] Why is `job_id`` stored as a plain `BIGINT`` in `candidate`` table instead of `@ManyToOne``?

Answer: In microservices, tables reside in separate databases (`job_db`` vs `candidate_db``). Foreign Key relationships cannot span different physical databases across services.

[Q21 (Angular)] What is Angular Standalone Components?

Answer: Introduced in Angular 14+, standalone components manage their own imports (`imports: [CommonModule, FormsModule, RouterLink]``) without requiring NgModule declarations.

[Q22 (Angular)] What is an Observable and RxJS in Angular?

Answer: An Observable represents a stream of asynchronous data events over time. Angular HttpClient methods return Observables which components subscribe to using `.subscribe()``.

[Q23 (Angular)] What is the purpose of `AuthService`` in Angular?

Answer: Centralizes user authentication state, handles login API calls, stores user session in `localStorage`, and provides `currentUser\$` stream to components.

[Q24 (Angular)] What is Angular Router Guard (`canActivate`)?

Answer: Interceptors that control page access based on user session roles (`employeeGuard`, `adminGuard`). If unauthorized, redirects user to `/login`.

[Q25 (Angular)] What is Two-Way Data Binding in Angular?

Answer: Syncs property values between Component TypeScript code and HTML template view automatically using `[(ngModel)]`.

[Q26 (Project)] How is duplicate candidate registration prevented in Candidate Service?

Answer: CandidateService checks `findByEmailIgnoreCase(email)` and `findByEmployeeId(empId)`. If either exists, throws 400 Bad Request **An employee with this email or employee ID already exists.**

[Q27 (Project)] Why is Employee ID NOT allowed for Employee Login?

Answer: To enforce strict single-email identity (`@company.com`). Logging in via Employee ID created identity ambiguities across multi-job application candidate rows.

[Q28 (Project)] How are notifications personalized for the logged-in employee?

Answer: Backend endpoint `GET /api/candidates/notifications/employee/{identifier}` resolves all candidate application IDs belonging to the employee email/empId and returns notifications strictly for those IDs.

[Q29 (Project)] What happens when an admin deletes a candidate record?

Answer: `CandidateService.deleteCandidate(id)` executes a `@Transactional` operation that deletes candidate notifications from `notification`, interviews from `interview`, and candidate profile from `candidate` table.

[Q30 (Project)] How does the system prevent applying for a closed job?

Answer: `CandidateService` calls `JobServiceClient.getJobById(jobId)`. If `job.getStatus()` is `CLOSED`, it throws a runtime exception and rejects the application.

10. 2-Minute Scripts & Night-Before Interview Cheat Sheet

10.1 'Tell Me About Your Project' (2-Minute Script for Each Member)

Aditya's 2-Minute Script (Focus: Architecture & Infrastructure):

'Our project is an Internal Recruitment System built with Spring Boot Microservices and Angular 17. I led the system architecture design. We implemented a microservices pattern with Spring Cloud Gateway running on port 8080 as our endpoint reverse-proxy and Netflix Eureka Server on port 8761 for dynamic service discovery. Each business service—job-service, candidate-service, and admin-service—owns a dedicated MySQL database following the database-per-service pattern. This ensures fault isolation, loose coupling, and independent microservice scalability.'

Khusboo's 2-Minute Script (Focus: Candidate Service & Registration/Auth):

'I focused on the Candidate Microservice, candidate registration, and authentication mechanics. I implemented employee registration with strict company email validation (@company.com) and duplicate protection against existing emails and employee IDs. I also refactored employee login to authenticate strictly via company email and password. Furthermore, I built our personalized notification engine which isolates in-app notifications per employee session without cross-employee data exposure.'

Naman's 2-Minute Script (Focus: Job/Admin Services & Frontend/Demo):

'I worked on the Job Microservice, Admin Microservice, and Angular frontend integration. I implemented job posting CRUD, job status toggling, and safe job deletion. On the admin side, I developed Designation Master data management and candidate interview scheduling (ONLINE Google Meet vs OFFLINE room location). On the frontend, I built our Angular standalone components and RxJS services styled with a professional LinkedIn dark blue theme.'

10.2 'Night Before Interview' 1-Page Revision Cheat Sheet

Component / Topic	Key Technical Details & Values to Remember
Service Discovery	Eureka Server Port: 8761 <code>@EnableEurekaServer</code> URL: <code>http://localhost:8761</code>
API Gateway	Spring Cloud Gateway Port: 8080 Routes: <code>/api/jobs/**</code> , <code>/api/candidates/**</code> , <code>/api/admin/**</code>
Job Service	Port: 8081 DB: <code>job_db</code> Table: <code>job_posting</code> Entity: <code>JobPosting.java</code>
Candidate Service	Port: 8082 DB: <code>candidate_db</code> Tables: <code>candidate</code> , <code>notification</code> , <code>interview</code>
Admin Service	Port: 8083 DB: <code>admin_db</code> Tables: <code>admin</code> , <code>designation</code> Admin Login: <code>admin@company.com</code>
Frontend App	Angular 17+ Port: 4200 Standalone Components Theme Colors: <code>#0A66C2</code> , <code>#0B2A4A</code> , <code>#F7F9FC</code>
Employee Login Rule	EMAIL ONLY (<code>@company.com</code>) + Password. Employee ID alone is NOT accepted for login.
Registration Rules	Requires First Name, Last Name, DOB, EmpID, Email (<code>@company.com</code>), Matching Passwords. Rejects duplicate email/empId.
Feign Client Call	<code>CandidateService</code> calls <code>JobServiceClient.getJobById(jobId)</code> -> <code>JOB-SERVICE</code> to verify OPEN status.
Database Schemas	3 separate schemas: <code>job_db</code> , <code>candidate_db</code> , <code>admin_db</code> (Database-per-Service pattern).